

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CÁTEDRA DE REDES DE COMPUTADORAS

TRABAJO PRÁCTICO Nº6

Grupo: “Los simuLANdores”

Alumnos:

Barrio, Rafael

Chesta, Santino

Garay, Alexis Tomás

Guzmán Gonzalez, Pedro

Vera, Agustín

Martin, Agostina Rocio

Zucchella Paz, Valentino

Profesor:

Ing. Santiago Martin Henn

Parte 1: Taller de armado y verificación de cables Cat5/Cat5e bajo estándar T568A/B.

1) Investigar los pasos para crimpar un cable bajo la norma T568A/B DERECHO. No cruzado. Ver Fig:

2) Construir 1 (un) cable por grupo, de 1 - 1.5 m. aproximadamente. Tipo DERECHO, NO CRUZADO.

3) VERIFICAR la buena construcción de un cable. Para ello, buscar otro grupo e intercambiar cables.

Realizarán:

a) Inspección visual para la calidad constructiva (sean extremadamente críticos) tomen fotos, indiquen que ven bien y que ven mal. Justifiquen.

b) Verificación eléctrica utilizando un tester para cables ethernet.

4) Documentar con que grupo intercambiaron cables (pueden intercambiar más de una vez, no importa, mientras no tengan su propio cable). Tomar fotografías que evidencien el proceso técnico de verificación.

Respuestas parte 1

Los pasos para crimpear son los siguientes: pelar la sección que debe ir dentro del capuchón, ordenar los cables siguiendo la forma de la imagen (usamos la T568B de la imagen 1) con la lengüeta del capuchón mirando abajo, introducirlos chequeando que la disposición en la punta sea pareja y finalmente crimpear con la herramienta.

Luego del armado, nos cercioramos el correcto funcionamiento usando un tester como en la imagen 2, deben prender en secuencia todos los números de ambos lados del tester, eso indica que no hay ningún corto y que todos los segmentos funcionan bien.

Repetimos la inspección cuidadosa y el test de funcionamiento con el grupo Bitless.

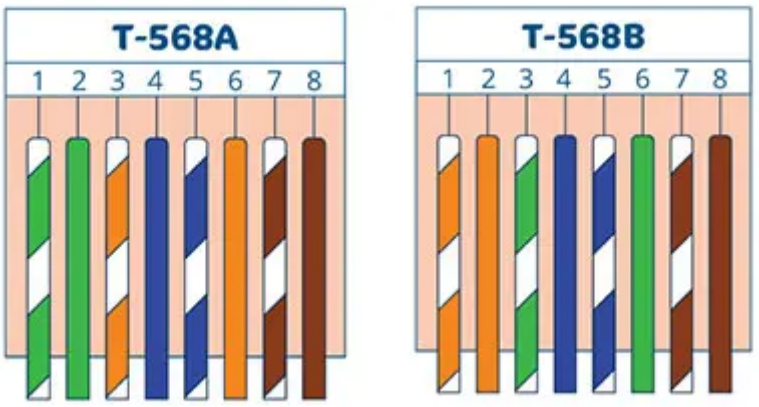


Imagen 1: conexión UTP derecho



Imagen 2: tester con UTP



Imagen 3: UTP de nuestro grupo

Parte 2: Equipamiento físico, verificación y utilización de equipos de red y generación de tráfico.

Levantar y testear una LAN

1) Configurar MANUALMENTE la interfaz de red de la computadora que elija el grupo para realizar la actividad.

a) Configurar la dirección IPv4.

b) Configurar la máscara de Subred.

c) Configurar la default gateway.

d) **Justificar las direcciones elegidas y si al final tenían o no sentido para la actividad.**

2) De a dos o más grupos por switch, conectarán una computadora por cada grupo al mismo utilizando los cables que tengan de la parte 1 de este TP. Por el amor de todo lo que es bueno espero que hayan verificado bien el cable, la cátedra no se hace responsable por interfaces de red reventadas. **Verificar que pueden llegar a la PC de otro grupo usando ping.**

3) Utilizando ncat, establecer un receptor UDP en una computadora:
ncat -u -l 5000

- Desde la segunda computadora, enviar mensajes al receptor:

ncat -u 192.168.1.10 5000

- Alternativamente podés usar scripts de Python, Packetsender, o lo que gustes para generar/recibir paquetes.

4) Capturar los paquetes recibidos desde la computadora del otro grupo con wireshark. Documentar.

5) Responder las siguientes preguntas:

a) ¿Se observa algún procedimiento de establecimiento de conexión antes de enviar los datos?

No. UDP es un protocolo no orientado a conexión, por lo que no realiza un handshake antes de transmitir datos, a diferencia de TCP. El emisor envía directamente los datagramas UDP a la dirección IP y puerto de destino.

Pero si podemos ver tráfico ARP antes del primer datagrama para resolver la dirección MAC de destino. Es un procedimiento de comunicación de red local, no un establecimiento de conexión UDP.

b) ¿Existe algún mecanismo de confirmación de recepción equivalente a los ACK de TCP?

No. UDP no implementa mecanismos de confirmación de recepción (ACK). Despues de enviar no hay confirmación de que se recibió, por lo que tampoco se hacen retransmisiones por pérdidas. Si se necesitan mecanismos de confirmación para una aplicación, ésta deberá implementarlos a nivel de aplicación.

c) ¿Qué ocurre si se envían mensajes mientras el receptor UDP no está ejecutándose?

Los paquetes pueden llegar al receptor, pero si no hay una aplicación escuchando el puerto, el sistema operativo los descarta.

En algunos casos, el receptor puede responder con un ICMP de tipo Destination Unreachable ó Port Unreachable, indicando que el puerto de destino no está disponible.

UDP no almacena los mensajes para entregarlos cuando la aplicación receptora vuelva a ejecutarse, ni intenta retransmitirlos automáticamente.

d) ¿UDP garantiza que los mensajes lleguen en el mismo orden en que fueron enviados?

No. UDP no garantiza el orden de entrega de los datagramas, los distintos retardos en la red para cada uno puede hacer que lleguen desordenados. Además no tienen orden de secuencia para reordenarlos.

e) ¿Qué aplicaciones reales utilizan UDP y por qué?

UDP se utiliza principalmente en aplicaciones donde se prioriza la baja latencia, la simplicidad o la eficiencia de transmisión frente a las garantías de entrega de TCP.

- DNS: Permite realizar consultas y obtener respuestas con poco intercambio de mensajes.
- VoIP: Prioriza la transmisión de voz en tiempo real, evitando demoras por retransmisiones.
- Juegos online: Permite actualizar rápidamente posiciones y estados de los jugadores.
- Streaming de video: Menor latencia y hay tolerancia de cierta pérdida de información.
- DHCP: Permite intercambiar mensajes de configuración de red incluso cuando el cliente todavía no tiene una dirección IP asignada.
- QUIC (HTTP/3): Utiliza UDP como base e implementa confiabilidad, control de congestión y seguridad en capas superiores.

Desarrollar un pequeño servidor

1) En una de las computadoras, crear un directorio que contenga al menos tres archivos de diferentes tipos (por ejemplo, TXT, PDF e imagen). Iniciar un servidor HTTP utilizando Python:
python3 -m http.server 8000

2) Desde la segunda computadora, acceder al servidor mediante un navegador web, utilizando la dirección IP correspondiente y el puerto 8000 (o el que hayas elegido). Descargar uno de los archivos disponibles.

3) Repetir la transferencia utilizando curl desde una terminal:
curl -O http://192.168.1.10:8000/archivo.pdf

4) Capturar el tráfico generado con Wireshark. ¿De qué tipo es?

No olvides volver a configurar tu interfaz de red para que obtenga direcciones de forma automática, si la dejas con la IP manual que usamos acá seguramente no puedas acceder a internet cuando lo intentes.

Respuestas parte 2

Levantar y testear LAN:

Configuramos las dirección IPv4 de Ethernet de los dos equipos en la misma subred local utilizando las IP privadas 192.168.1.10 y 192.168.1.1. Se aplicó la máscara de subred 255.255.255.0 (/24), definiendo los primeros 24 bits para la porción de red (192.168.1.0/24) y los primeros últimos 8 bits para identificar a cada host. De esta manera, ambos equipos pertenecen al mismo segmento LAN y pueden comunicarse directamente sin necesidad de pasar por un gateway o router.

Para la configuración, en Windows 11 es sencillo:
Click derecho al Wifi > Configuración de Red e Internet > Ethernet > Asignación de IP (Editar) > Cambiar a manual > Tildar IPv4 > Llenar formulario



Para Linux (CachyOS/Arch) son algunos comandos un poco más complicados pero con ayuda de IA se volvió relativamente sencillo:

```
sudo nmcli connection add \
   type ethernet \
   ifname eno1 \ # <---------- poner nombre de LAN correcto
   con-name tp-redes \
   ipv4.method manual \
   ipv4.addresses 192.168.1.10/24 \
   ipv4.never-default yes \
   ipv6.method disabled \
   connection.autoconnect no
```

Luego activamos la conexión con:

```
sudo nmcli connection up tp-redes
```

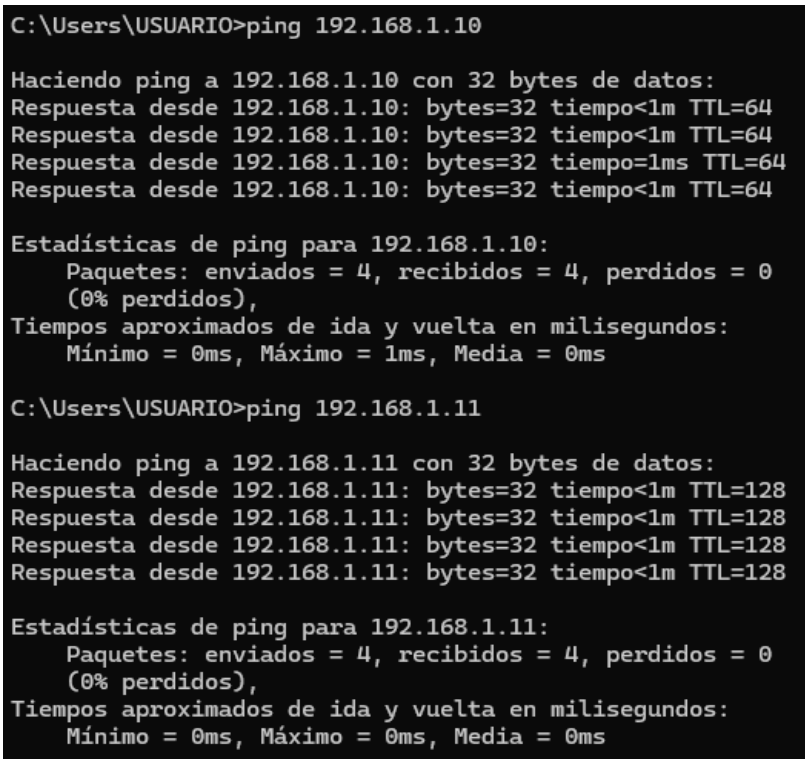
Justificación de configuración de IP:

Se dejaron sin configurar la puerta de enlace predeterminada (Default Gateway) y el sistema de nombres de dominio (DNS), debido a sus características.

- El gateway predeterminado permite alcanzar redes diferentes de la subred local, normalmente mediante un router
- Como la actividad solo requiere comunicar 2 computadoras en la misma LAN, no hace falta configurar un gateway
- El DNS sirve para resolver nombres de dominio a direcciones IP. Tampoco hace falta para comunicarse directamente mediante direcciones IP.

No obstante, estos parámetros serán indispensables en un escenario de red extendido, osea con salida a internet.
Si bien en un entorno LAN pequeño basta con usar direcciones IP directas, contar con un servidor DNS local puede ser de utilidad, por ejemplo:
Permite asociar nombres simples (ej. servidor.local) a los equipos, lo que facilita la administración y evita fallas si cambia una dirección IP.

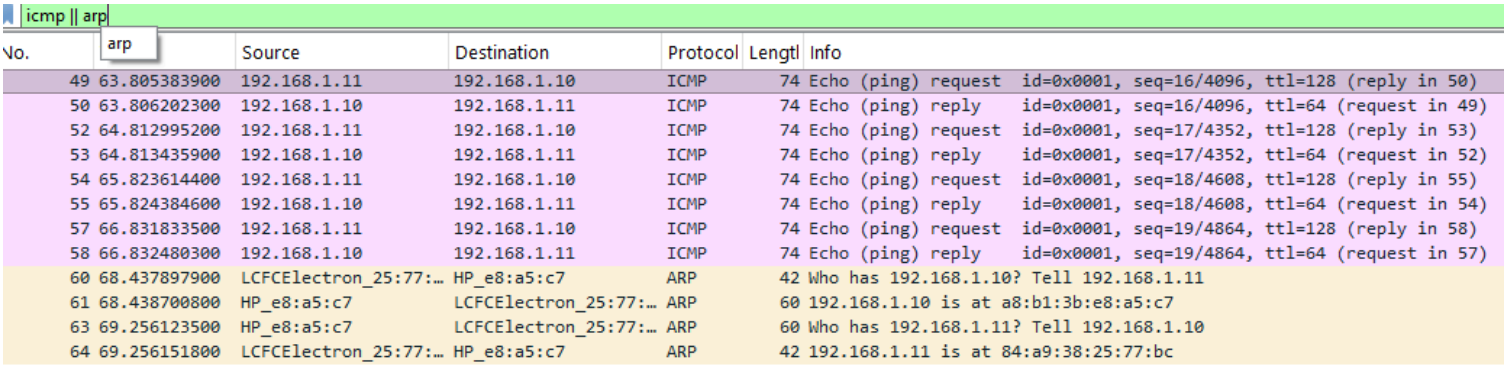
Luego de la configuración mandamos pings de una computadora a otra usando el switch. En la imagen vemos los pings desde .10 a ella misma y a .11.



Luego, usando packet sender pudimos mandar UTP de una hacia la otra. En la imagen se ve como una compu (.11) recibe los paquetes en el puerto 5000.

From IP	From Port	To Address	To Port	Method	Error	ASCII	Hex
192.168.1.10	49257	You	5000	UDP		aguante CAB + CABJ \r	61 67 75 61 6E 74 65 20 43 41 42 20 2B 20 43 41 42 4A 20 0D
192.168.1.10	49257	You	5000	UDP		aguante CAB + CABJ \r	61 67 75 61 6E 74 65 20 43 41 42 20 2B 20 43 41 42 4A 20 0D

Además, **analizamos con wireshark los paquetes del ping y el contenido de los paquetes UDP**.
En el ping capturamos el siguiente tráfico:



icmp || arp

No.	Time	Source	Destination	Protocol	Length	Info
49	63.805383900	192.168.1.11	192.168.1.10	ICMP	74	Echo (ping) request id=0x0001, seq=16/4096, ttl=128 (reply in 50)
50	63.806202300	192.168.1.10	192.168.1.11	ICMP	74	Echo (ping) reply id=0x0001, seq=16/4096, ttl=64 (request in 49)
52	64.812995200	192.168.1.11	192.168.1.10	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (reply in 53)
53	64.813435900	192.168.1.10	192.168.1.11	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=64 (request in 52)
54	65.823614400	192.168.1.11	192.168.1.10	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=128 (reply in 55)
55	65.824384600	192.168.1.10	192.168.1.11	ICMP	74	Echo (ping) reply id=0x0001, seq=18/4608, ttl=64 (request in 54)
57	66.831833500	192.168.1.11	192.168.1.10	ICMP	74	Echo (ping) request id=0x0001, seq=19/4864, ttl=128 (reply in 58)
58	66.832480300	192.168.1.10	192.168.1.11	ICMP	74	Echo (ping) reply id=0x0001, seq=19/4864, ttl=64 (request in 57)
60	68.437897900	LCFCElectron_25:77:…	HP_e8:a5:c7	ARP	42	Who has 192.168.1.10? Tell 192.168.1.11
61	68.438700800	HP_e8:a5:c7	LCFCElectron_25:77:…	ARP	60	192.168.1.10 is at a8:b1:3b:e8:a5:c7
63	69.256123500	HP_e8:a5:c7	LCFCElectron_25:77:…	ARP	60	Who has 192.168.1.11? Tell 192.168.1.10
64	69.256151800	LCFCElectron_25:77:…	HP_e8:a5:c7	ARP	42	192.168.1.11 is at 84:a9:38:25:77:bc

Podemos ver como hay paquetes ICMP y ARP:
ICMP: los mensajes de solicitud y respuesta del ping.
ARP: los mensajes utilizados para averiguar o confirmar qué MAC corresponde a una dirección IP dentro de la LAN.

Si analizamos que desde la compu .11 se hizo ping a .10:

Un ping consiste en enviar un mensaje ICMP Echo Request (compu .11) y esperar un ICMP Echo Reply (compu .10). Como en el ping (comando windows) se mandan 4 pings, lo normal es encontrar 4 Echo Request y 4 Echo Reply, es decir, 8 mensajes ICMP, suponiendo que todos los pings reciban respuesta, como vemos en la imagen.

Yendo a los ARP, la pregunta es por qué aparecen después:

Porque los sistemas operativos mantienen una caché ARP (también llamada tabla de vecinos). Lo más probable es que las computadoras ya conocieran las MAC mediante sus cachés y posteriormente estuvieran verificando o renovando esas asociaciones.

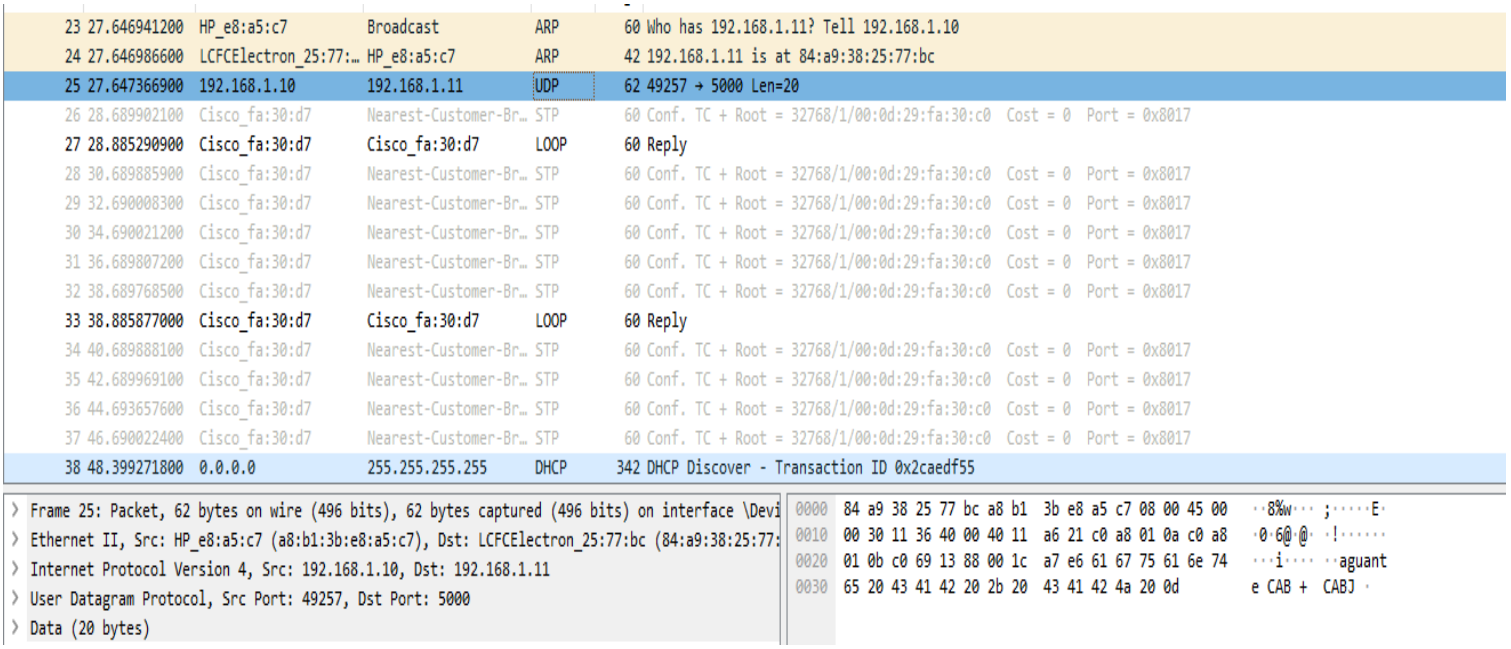
Ahora analizando los UDP (trafico sin filtros de wireshark):

Vemos que solo hay un ARP antes de mandar el paquete (esto es porque hicimos el envio apenas conectamos los cables al switch, entonces no tenían nada en caché). LOOP y STD son paquetes de deteccion de bucles por parte del switch, no analizamos. DHCP es para busqueda y exploración de IP, lo hace el Switch y tampoco analizamos.

UDP, no se observa ningún establecimiento de conexión UDP ni una confirmación ACK propia de UDP. El datagrama se transmite directamente una vez que la red dispone de la información necesaria para entregarlo.

Además el puerto origen 49257 es el que usó la compu que mandó por PacketSender, mientras que 5000 es el puerto del receptor UDP. La longitud coincide como también el mensaje enviado:

$$28\ bytesUDP\ =\ 8\ bytesdecabecera\ +\ 20\ bytesdedatos$$



El punto 5 se contestó en el lugar de la consigna

Desarrollar un pequeño servidor:

En una de las computadoras, crear un directorio que contenga al menos tres archivos de diferentes tipos (por ejemplo, TXT, PDF e imagen). Iniciar un servidor HTTP utilizando Python, en una carpeta con los archivos ejecutamos por terminal:

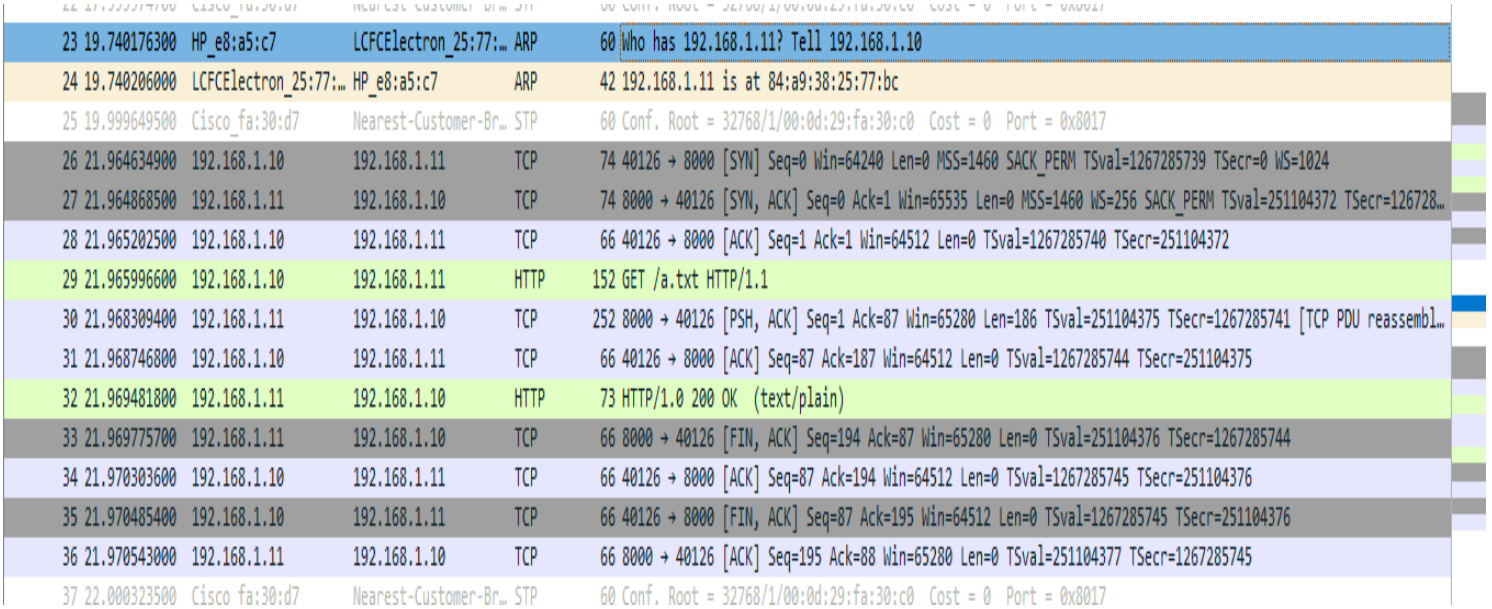
```
python3 -m http.server 8000
```

Luego desde la otra compu entramos a http://192.168.1.11:8000 y descargamos el archivo de texto, por último repetimos con curl.



```
3TO AÑO/REDES DE COMPUTADORAS/tp6-test-python-server took 4m39s
❯ curl -O http://192.168.1.11:8000/a.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100     7 100     7    0     0   1116      0                              0
```

Por ultimo podemos analizar el trafico capturado con estos CURL, analizamos el tráfico TCP ya que es una conexión HTTP/1 (la cual se hace usando TCP):



23	19.740176300	HP_e8:a5:c7	LCFCElectron_25:77:…	ARP	60 Who has 192.168.1.11? Tell 192.168.1.10
24	19.740206000	LCFCElectron_25:77:…	HP_e8:a5:c7	ARP	42 192.168.1.11 is at 84:a9:38:25:77:bc
25	19.999649500	Cisco_fa:30:d7	Nearest-Customer-Br…	STP	60 Conf. Root = 32768/1/00:0d:29:fa:30:c0 Cost = 0 Port = 0x8017
26	21.964634900	192.168.1.10	192.168.1.11	TCP	74 40126 → 8000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1267285739 TSecr=0 WS=1024
27	21.964868500	192.168.1.11	192.168.1.10	TCP	74 8000 → 40126 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=251104372 TSecr=126728…
28	21.965202500	192.168.1.10	192.168.1.11	TCP	66 40126 → 8000 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1267285740 TSecr=251104372
29	21.965996600	192.168.1.10	192.168.1.11	HTTP	152 GET /a.txt HTTP/1.1
30	21.968309400	192.168.1.11	192.168.1.10	TCP	252 8000 → 40126 [PSH, ACK] Seq=1 Ack=87 Win=65280 Len=186 TSval=251104375 TSecr=1267285741 [TCP PDU reassembl…
31	21.968746800	192.168.1.10	192.168.1.11	TCP	66 40126 → 8000 [ACK] Seq=87 Ack=187 Win=64512 Len=0 TSval=1267285744 TSecr=251104375
32	21.969481800	192.168.1.11	192.168.1.10	HTTP	73 HTTP/1.0 200 OK (text/plain)
33	21.969775700	192.168.1.11	192.168.1.10	TCP	66 8000 → 40126 [FIN, ACK] Seq=194 Ack=87 Win=65280 Len=0 TSval=251104376 TSecr=1267285744
34	21.970303600	192.168.1.10	192.168.1.11	TCP	66 40126 → 8000 [ACK] Seq=87 Ack=194 Win=64512 Len=0 TSval=1267285745 TSecr=251104376
35	21.970485400	192.168.1.10	192.168.1.11	TCP	66 40126 → 8000 [FIN, ACK] Seq=87 Ack=195 Win=64512 Len=0 TSval=1267285745 TSecr=251104376
36	21.970543000	192.168.1.11	192.168.1.10	TCP	66 8000 → 40126 [ACK] Seq=195 Ack=88 Win=65280 Len=0 TSval=251104377 TSecr=1267285745
37	22.000323500	Cisco_fa:30:d7	Nearest-Customer-Br…	STP	60 Conf. Root = 32768/1/00:0d:29:fa:30:c0 Cost = 0 Port = 0x8017

Diremos **cliente** a la compu que quiere acceder al recurso (192.168.1.11:40216) y **servidor** a la que muestra los archivos de su computadora (192.168.1.11:8000)

Podemos ver, además de los ARP para confirmar la IP al comienzo, que el ciclo de una request es el siguiente:

1. Establecimiento de conexión: Three-Way Handshake.
2. Transferencia de datos: solicitud HTTP, respuesta HTTP y confirmaciones TCP.
3. Cierre de conexión: intercambio de FIN y ACK.

El 3 way handshake para iniciar la conexión (paquetes de 26 a 28):

1. El cliente manda SYN: quiere iniciar la conexión, iniciar la sincronización de números de secuencia.
2. El servidor responde SYN+ACK: también quiere iniciar conexión, comunica su número de secuencia y confirma recepción de SYN del cliente.
3. El cliente responde con ACK: confirma que recibió el SYN del servidor.

A partir de este momento, la conexión TCP queda establecida y ambos extremos pueden intercambiar datos. Estos son los paquetes 29 a 32.

TCP ya estableció la conexión y ahora HTTP puede utilizarla para solicitar y transferir el archivo `a.txt`, ¿cómo?

- HTTP define qué recurso solicita el cliente y cómo responde el servidor.

- TCP transporta los bytes de esos mensajes, los ordena y permite confirmar su recepción.

Paquete 29 — HTTP/1.1 GET /a.txt: petición para obtener recurso.

El mensaje HTTP viaja dentro de un segmento TCP, que a su vez viaja dentro de un paquete IP y de una trama Ethernet.

Paquete 30 y 31:
- 30: Servidor responde con ACK (recibí tu paquete) y PSH (indica a TCP los datos se deben entregar sin demoras)
- 31: Cliente responde con ACK (recibí tu paquete)

Luego, en el paquete 32 el servidor envía la respuesta: un HTTP/1.0 con código 200 (OK) y el contenido del recurso solicitado como texto plano (por ser .txt), se puede ver las cabeceras y contenido en la parte de Hypertext Transfer Protocol de wireshark

Por ultimo, viene el 4 way handshake para terminal la conexión TCP (paquetes de 33 a 36):
1. El servidor manda FIN y ACK: terminó de enviar datos y quiere cerrar su dirección de transmisión.
2. El cliente responde ACK: recibí tu FIN, servidor.
3. El cliente responde FIN y ACK: indica que también termina su dirección de transmisión y envía su propio FIN.
4. El servidor envia ACK: confirma el FIN del cliente.

Con esto se completa el intercambio de cierre. El cliente que envió el último FIN normalmente atraviesa el estado `TIME_WAIT` antes de liberar completamente su conexión.